Vel Tech Group of Institutions

Name: GOPICHAND KAKIVAI
Email: vtu29748@veltech.edu.in
Roll no: vtu29748
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

2028_NeoColab_Design and Analysis of AlgorithmsS1L3

VTU_DAA_Week 10_CY

Attempt : 1
Total Mark : 50
Marks Obtained : 42

Section 1 : Coding

1. Problem Statement

Meera, the director of a national disaster response team, must deploy three specialized rescue units to three critical disaster sites. Each rescue unit has a different response time (in minutes) to reach each site due to distance, terrain, and accessibility constraints.

However, Meera has an additional operational rule:

Rescue Unit 1 cannot be assigned to Site 3 due to equipment limitations.Rescue Unit 2 must not be assigned to Site 1 because of route restrictions.

Meera decides to use the Branch and Bound technique to explore only valid and efficient assignments while respecting all constraints.

Your task is to determine the total response time of the best valid

deployment plan that satisfies all given restrictions.

Input Format

The input consists of three lines with three space-separated integers each, representing a 3 × 3 response time matrix.

Each cell (i, j) represents the response time (in minutes) if the ith rescue unit is deployed to the jth disaster site.

Output Format

The output should display the total response time in the following exact format: "Total Response Time: X" ,where X is the total response time of the best valid deployment plan.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 12 25 40
30 14 28
16 18 11
Output: Total Response Time: 37

Answer

```
#include <stdio.h>
#include <limits.h>

int cost[3][3];
int used[3];
int minCost = INT_MAX;

void solve(int unit, int currCost) {
    if (unit == 3) {
        if (currCost < minCost)
            minCost = currCost;
        return;
    }

    if (currCost >= minCost)
```

```
        return;

    for (int site = 0; site < 3; site++) {
        if (used[site]) continue;

        if (unit == 0 && site == 2) continue;
        if (unit == 1 && site == 0) continue;

        used[site] = 1;
        solve(unit + 1, currCost + cost[unit][site]);
        used[site] = 0;
    }
}

int main() {
    for (int i = 0; i < 3; i++)
        for (int j = 0; j < 3; j++)
            scanf("%d", &cost[i][j]);

    solve(0, 0);

    printf("Total Response Time: %d", minCost);

    return 0;
}
```

Status : Correct **Marks : 10/10**

2. Problem Statement

Captain Aidan, a daring pirate, has discovered an ancient treasure cave filled with valuable relics. However, the cave is protected by an ancient spell that limits the weight of the treasure he can carry. Aidan has a magic knapsack with a maximum weight capacity of W. Inside the cave, he finds n different treasures, each with a specific weight and value.

Since he can only take each treasure once, Aidan must carefully choose which items to take to maximize the total value without exceeding the weight limit of his knapsack.

Help Captain Aidan pick the most valuable treasures while staying within the weight limit using branch and bound technique

Input Format

The first line contains two integers n (the number of items) and W (the capacity of the knapsack), separated by a space.

The second line contains n integers, where the i-th integer represents the weight of the i-th item.

The third line contains n integers, where the i-th integer represents the value of the i-th item

Output Format

The output prints the single integer representing the maximum value obtained by filling the knapsack with the given set of items.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5 60
10 20 30 40 50
60 100 120 140 180
Output: 280

Answer

```
#include <stdio.h>
#include <stdlib.h>

typedef struct {
    int wt, val;
    double ratio;
} Item;

int cmp(const void *a, const void *b) {
    Item *i1 = (Item *)a;
    Item *i2 = (Item *)b;
    if (i1->ratio < i2->ratio) return 1;
```

```
    if (i1->ratio > i2->ratio) return -1;
    return 0;
}

double bound(int n, int W, int idx, int wt, int val, Item items[]) {
    double profit = val;
    int w = wt;

    for (int i = idx; i < n; i++) {
        if (w + items[i].wt <= W) {
            w += items[i].wt;
            profit += items[i].val;
        } else {
            profit += (W - w) * items[i].ratio;
            break;
        }
    }
    return profit;
}

int maxProfit = 0;

void knapsack(int n, int W, int idx, int wt, int val, Item items[]) {
    if (wt <= W && val > maxProfit)
        maxProfit = val;

    if (idx == n)
        return;

    if (bound(n, W, idx, wt, val, items) <= maxProfit)
        return;

    if (wt + items[idx].wt <= W)
        knapsack(n, W, idx + 1, wt + items[idx].wt, val + items[idx].val, items);

    knapsack(n, W, idx + 1, wt, val, items);
}

int main() {
    int n, W;
    scanf("%d %d", &n, &W);
```

```
    Item items[n];

    for (int i = 0; i < n; i++)
        scanf("%d", &items[i].wt);

    for (int i = 0; i < n; i++) {
        scanf("%d", &items[i].val);
        items[i].ratio = (double)items[i].val / items[i].wt;
    }

    qsort(items, n, sizeof(Item), cmp);

    knapsack(n, W, 0, 0, 0, items);

    printf("%d", maxProfit);

    return 0;
}
```

Status : Correct **Marks : 10/10**

3. Problem Statement

Commander Vikram is preparing a spacecraft for an interplanetary research mission. The spacecraft has a limited cargo capacity, so he must carefully decide which scientific equipment modules to load.

Each module has a scientific importance value and a specific weight. Commander Vikram wants to load the spacecraft in such a way that the total scientific value of the selected modules is maximized without exceeding the cargo weight limit.

To solve this efficiently, he uses the Branch and Bound technique applied to the 0/1 Knapsack problem, which explores possible combinations of equipment and eliminates options that cannot lead to better solutions.

Your task is to help Commander Vikram determine the maximum achievable value and the selected module weights that should be loaded into the spacecraft.

Note: The weights must be printed in reverse order of the input, meaning the algorithm starts evaluating items from the last item and prints the selected weights first.

Input Format

The first line contains an integer n, representing the number of equipment modules available.

The second line contains n space-separated integers val[i], representing the scientific value of each module.

The third line contains n space-separated integers wt[i], representing the weight of each module.

The fourth line contains an integer W, representing the maximum cargo capacity of the spacecraft.

Output Format

The first line output displays "Maximum value: " followed by an integer, representing the maximum value that can be achieved.

The second line output displays "Selected weights: " followed by space-separated integers, printed in reverse order of the input.

Refer to the sample outputs for the exact format.

Sample Test Case

Input: 3
60 100 120
10 20 30
50
Output: Maximum value: 220
Selected weights: 30 20

Answer

```
#include <stdio.h>
```

```
int n, W;
int val[10], wt[10];
int bestSet[10], currSet[10];
int maxProfit = 0;

double bound(int idx, int weight, int profit) {
    double b = profit;
    int w = weight;

    for (int i = idx; i < n; i++) {
        if (w + wt[i] <= W) {
            w += wt[i];
            b += val[i];
        } else {
            b += (double)(W - w) * val[i] / wt[i];
            break;
        }
    }
    return b;
}

void knapsack(int idx, int weight, int profit) {
    if (weight <= W && profit > maxProfit) {
        maxProfit = profit;
        for (int i = 0; i < n; i++)
            bestSet[i] = currSet[i];
    }

    if (idx == n) return;

    if (bound(idx, weight, profit) <= maxProfit) return;

    if (weight + wt[idx] <= W) {
        currSet[idx] = 1;
        knapsack(idx + 1, weight + wt[idx], profit + val[idx]);
        currSet[idx] = 0;
    }

    knapsack(idx + 1, weight, profit);
}

int main() {
```

```
    scanf("%d", &n);

    for (int i = 0; i < n; i++)
        scanf("%d", &val[i]);

    for (int i = 0; i < n; i++)
        scanf("%d", &wt[i]);

    scanf("%d", &W);

    for (int i = 0; i < n; i++) {
        currSet[i] = 0;
        bestSet[i] = 0;
    }

    knapsack(0, 0, 0);

    printf("Maximum value: %d\n", maxProfit);
    printf("Selected weights:");

    for (int i = n - 1; i >= 0; i--) {
        if (bestSet[i])
            printf(" %d", wt[i]);
    }

    return 0;
}
```

Status : Partially correct ***Marks : 2/10***

4. Problem Statement

You are tasked with solving the Travelling Salesman Problem (TSP) for a given set of cities.

Given the distances between pairs of cities, your goal is to find the shortest route that visits each city exactly once and returns to the starting city.

Note: Solve it using Branch and Bound approach.

Input Format

The first line of input contains an integer n, representing the number of cities.

The following n lines each contain n space-separated integers, representing the distances between pairs of cities.

The distance between city i and city j is given at the ith row and jth column of the matrix.

Output Format

The first line displays "The Path is: " followed by the sequence of cities visited in the shortest path,

separated by "--->".

The second line displays "Minimum cost is X", where X is the minimum cost of the shortest path found.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4
0 4 1 3
4 0 2 1
1 2 0 5
3 1 5 0
Output: The Path is: 1--->3--->2--->4--->1
Minimum cost is 7

Answer

```
#include <stdio.h>
#include <limits.h>

int n;
int cost[10][10];
int visited[10];
int currPath[11], bestPath[11];
int minCost = INT_MAX;
```

```
void tsp(int curr, int count, int currCost) {
    if (count == n) {
        if (cost[curr][0] != 0) {
            int total = currCost + cost[curr][0];
            if (total < minCost) {
                minCost = total;
                for (int i = 0; i < n; i++)
                    bestPath[i] = currPath[i];
                bestPath[n] = 0;
            }
        }
        return;
    }

    for (int i = 0; i < n; i++) {
        if (!visited[i] && cost[curr][i] != 0) {
            int temp = currCost + cost[curr][i];
            if (temp >= minCost)
                continue;

            visited[i] = 1;
            currPath[count] = i;
            tsp(i, count + 1, temp);
            visited[i] = 0;
        }
    }
}

int main() {
    scanf("%d", &n);

    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            scanf("%d", &cost[i][j]);

    for (int i = 0; i < n; i++)
        visited[i] = 0;

    visited[0] = 1;
    currPath[0] = 0;
```

```
    tsp(0, 1, 0);

    printf("The Path is: ");
    for (int i = 0; i <= n; i++) {
        printf("%d", bestPath[i] + 1);
        if (i < n)
            printf("--->");
    }

    printf("\nMinimum cost is %d", minCost);

    return 0;
}
```

Status : Correct **Marks : 10/10**

5. Problem Statement

You are a logistics manager responsible for planning delivery routes for a fleet of vehicles. Each vehicle needs to visit a set of locations to deliver goods, and the cost of traveling between locations is known.

Your goal is to find the shortest route for each vehicle, ensuring that each location is visited exactly once before returning to the starting point.

Note: Solve it using Branch and Bound approach.

Input Format

The first line contains an integer n, the number of locations.

The next n lines contain n integers each, representing the cost matrix of traveling between locations.

The ith integer on the jth line represents the cost of traveling from location i to location j.

Output Format

The first line of output displays "Shortest Path: " followed by the shortest path that visits all cities.

The second line of output displays "Minimum cost is " followed by the minimum cost.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5
0 10 8 9 7
10 0 10 5 6
8 10 0 8 9
9 5 8 0 6
7 6 9 6 0
Output: Shortest Path: 1 3 4 2 5 1
Minimum cost is 34

Answer

```
#include <stdio.h>
#include <limits.h>

int n;
int cost[10][10];
int visited[10];
int currPath[11], bestPath[11];
int minCost = INT_MAX;

void tsp(int curr, int count, int currCost) {
    if (count == n) {
        int total = currCost + cost[curr][0];
        if (total < minCost) {
            minCost = total;
            for (int i = 0; i < n; i++)
                bestPath[i] = currPath[i];
            bestPath[n] = 0;
        }
        return;
    }

    for (int i = 0; i < n; i++) {
        if (!visited[i]) {
```

```
            int temp = currCost + cost[curr][i];
            if (temp >= minCost)
                continue;

            visited[i] = 1;
            currPath[count] = i;
            tsp(i, count + 1, temp);
            visited[i] = 0;
        }
    }
}

int main() {
    scanf("%d", &n);

    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            scanf("%d", &cost[i][j]);

    for (int i = 0; i < n; i++)
        visited[i] = 0;

    visited[0] = 1;
    currPath[0] = 0;

    tsp(0, 1, 0);

    printf("Shortest Path: ");
    for (int i = 0; i <= n; i++)
        printf("%d ", bestPath[i] + 1);

    printf("\nMinimum cost is %d", minCost);

    return 0;
}
```

Status : Correct **Marks : 10/10**